Purpose of AI Use: I used Claude to write clear instructions for a coding agent so it builds exactly my changes for US-06: the database for job applications (issue #25) and the apply flow in the UI (issue #27, Task 6.3). I also used it to plan the database design and compare options before deciding.

Chat Link or Prompt/Response:

Prompt 1: [uploaded screenshots of my UI designs for the task]

Your role is to be a senior lead software developer with a PHD is software engineering, computer science and prompt engineering with experience developing web and mobile applications.

Context: CareerConnect, FastAPI + SQLAlchemy 2.0 + Pydantic v2 + PostgreSQL (Supabase) backend, React 19 + TypeScript + Vite frontend. Working on issue #25 (parent: #24, US-06): As a job seeker, I want to apply to a job posting by selecting one of my resumes so that I can submit applications directly through the platform.

Task: Design the database schema for job applications. An application links a job posting, an applicant, and the selected resume, and records a submission date and a status. On every listing that is applicable there will be 2 buttons "Select" and "Apply" for the job seeker. The Apply button must be in a more washed out colour by default and unclickable until the user presses the select button. This button as seen in the images above will open a vertical menu with the job seeker's resumes (resume1 and resume2 are example names and there can be more than 2 and they display the names of each resume the user has). The user selects 1 and once the resume for the job is selected the button name of "select" is replaced with the resume's name indicating it has been selected. Once this step is complete the Apply button returns to its default brighter colour indicating it may now be pressed for the user to apply to the listing with the selected resume.

Constraints:

- Look at how the project already stores users and resumes, and set up applications the same way. The code for this sprint is in the Sprint 2 backend folder.
- The job listings page and the job postings themselves haven't been built yet, so please don't create them. Just prepare the application feature now so it is ready to be connected as soon as job postings exist. In your plan, tell me what the simple follow-up step will be at that point.
- Even though job postings don't exist yet, duplicate protection should already work: one person must never be able to apply to the same posting twice.

- Applications need to stay properly connected to the real user and the real resume they came from.
- Don't change how users or resumes work today. If you need to touch an existing file, keep it to the smallest possible addition and tell me why in your plan.
- Keep this task to the data side only: no new API endpoints, no buttons or screens, no sample data. The Select and Apply behaviour I described is for context and will be built in a later task. The "selected but not applied yet" state only lives on screen and doesn't need to be saved. Only the final choice is saved when the person presses Apply.
- Don't open, show, or change any .env or secrets file.
- Don't add any new tools or libraries without asking me first.
- Where there is more than one reasonable way to do something, don't just pick one quietly. Give me the options, say which you recommend, and explain why. Three that I already know about: what should happen to an application if the person later deletes the resume they applied with, what should happen to applications if a user account is deleted, and what the status values should be (the README mentions Applied, Interview, Offered, and Rejected).

Acceptance criteria:

- Applications store references to the job posting, the applicant, and the selected resume
- Applications store a submission date (set automatically on creation) and a status (with a sensible default, e.g. "submitted")
- A uniqueness rule prevents the same applicant from having two applications to the same posting
- Foreign keys to job, user, and resume are enforced, and migration/schema file(s) are included
- Naming and style match the existing schema

First: propose a short plan and list the files you'd change. Wait for my approval before writing code.

Result 1:

Planning the approach before exploring the repository.

Planning the approach before exploring the repository.

I'll look at how the Sprint 2 backend stores users and resumes, then write up the plan. I won't write any code until you approve it.

Ran a command on your computer, updated tasks

I've read the Sprint 2 backend and haven't changed anything yet. I never opened `.env`.

Below are the plan and six decisions, each with a recommendation.

What I found

- **No migration tool.** `app.py` runs `Base.metadata.create_all` on startup. That creates missing tables but never alters existing ones. Alembic isn't in `requirements.txt`.
- **Style to copy:**
  - Old-style `Column(...)` models, plural snake_case table names, and `id` as an indexed primary key.
  - Foreign keys written as `ForeignKey("users.id")` with no ON DELETE rule.
  - `DateTime, default=datetime.utcnow`.
  - `role` stored as `String(20)` and validated by a Pydantic `Literal`.
  - Relationships declared by string name, and one model per file re-exported from `models/__init__.py`.
- **Resume deletion order.** `delete_resume` removes the file from disk first, then deletes the database row. This matters for decision 1.

Proposed `applications` table

Column	Type	Rule
`id`	Integer	primary key, indexed
`user_id`	Integer	NOT NULL, FK → `users.id`, ON DELETE CASCADE
`job_id`	Integer	NOT NULL, indexed, FK → `jobs.id` added later (decision 4)
`resume_id`	Integer	NOT NULL, FK → `resumes.id`, ON DELETE RESTRICT
`submitted_at`	DateTime	NOT NULL, defaults to `utcnow`
`status`	String(20)	NOT NULL, default `applied`, CHECK against the allowed values

The table also has `UNIQUE (user_id, job_id)`, named `uq_applications_user_id_job_id`. Relationships are `Application.applicant` and `Application.resume`, in one direction only, so `User` and `Resume` stay untouched.

Decisions

1. Resume deleted after applying

Option	Effect	Cost

A. RESTRICT (recommended)	The database refuses to delete a resume used by any application.	The delete button needs a clear “used in N applications” message.
B. SET NULL plus a saved filename	Deleting works as today. The application keeps the resume’s name as text but loses the file.	`resume_id` becomes nullable and a `resume_filename` column is added. Recruiters can’t open the resume.
C. CASCADE	Deleting a resume deletes its applications.	Silently loses job-search history. Not recommended.

I recommend A because you want applications to stay tied to the real resume, and recruiters will presumably need to open that file. Starting strict is also the easier direction. A to B later is one ALTER. B to A is hard once NULL rows exist.

Under A, today’s `delete_resume` would remove the file and then fail on commit, leaving a row with no file. The apply task must add a pre-check before `os.remove` in `routers/resume.py`. I’m not touching that router now. No applications can exist yet, so resume behaviour is unchanged today.

2. User account deleted

Option	Effect	Cost
A. CASCADE (recommended)	Their applications are deleted with the account.	Recruiters lose those records.
B. RESTRICT	Users with applications can’t be deleted.	Matches `resumes` today, but any account deletion needs manual cleanup.
C. SET NULL	Applications are kept anonymised.	`user_id` becomes nullable, so applications no longer link to a real user.

I recommend A because applications are the applicant’s personal data and should go with the account. There is no delete-user feature today. A future one would also have to delete in the order applications → resumes → user, because `resumes.user_id` has no ON DELETE rule.

3. Status values

Option	Values	Note
A. README's four, lowercase (recommended)	`applied` (default), `interview`, `offered`, `rejected`	Matches the README, and the lowercase style of `role` values like `job_seeker`.
B. Issue's default	`submitted`, then the README's other three	Two words for the same first state.
C. Longer pipeline	adds `under_review` and `withdrawn`	More realistic, but speculative before recruiter workflows exist.

I'd enforce the values with a database CHECK constraint, which is slightly stricter than `role`'s Pydantic-only check. Adding a status later means editing the model and running one ALTER on Supabase. Note that the default is `applied`, not the issue's example `submitted`.

4. Job reference before `jobs` exists

SQLAlchemy can't create a foreign key to a table it doesn't know about. Declaring `ForeignKey("jobs.id")` now would crash backend startup.

- **A. Plain NOT NULL `job_id` now (recommended).** The uniqueness rule is live immediately. Nothing is at risk before the foreign key exists, because no endpoint can write rows yet.
- B. Declare the foreign key but don't register the model. No table is created, so duplicate protection isn't live.
- C. Create a stub `jobs` table. You said not to, and it would clash with the real `Job` model.

Under A, the "foreign key to job is enforced" criterion is only met once step 1 of the follow-up below is done.

5. Resume must belong to the applicant

A foreign key to `resumes` only proves the resume exists, not that it's the applicant's.

- **A. Check in the apply endpoint (recommended).** The `_get_owned_resume` helper in `routers/resume.py` already does this check, so the endpoint can reuse it.
- B. Composite foreign key in the database. This needs a unique constraint added to the existing `resumes` table (one line in `resume.py` plus an ALTER on the shared database). It's the strongest guarantee, but it touches an existing table, so it's your call.

6. Migration files

- **A. Model plus plain SQL files (recommended).** `create_all` builds the table on startup. The SQL files make the table definition reviewable and runnable in the Supabase SQL editor.
- B. Model only.
- C. Alembic. This is a new dependency and needs a baseline of the existing tables, so I'll only do it if you say yes.

Job postings deleted later. I'd use RESTRICT, so postings with applicants get closed or archived rather than deleted. That rule goes into the prepared follow-up SQL file. Tell me if you'd rather have CASCADE.

Files

New:

- `Sprint 2/backend/models/application.py`
- `Sprint 2/backend/migrations/001_create_applications.sql` creates the same table as the model.
- `Sprint 2/backend/migrations/002_add_applications_job_fk.sql` is prepared now and run once the jobs table exists.

Existing file touched: `models/__init__.py` only, with one import line and one entry in `__all__`. `app.py` reaches models only through that package, so without this `create_all` would never see the new table.

Unchanged: `user.py`, `resume.py`, `app.py`, `database.py`, `requirements.txt`, all routers, and the frontend. New files will use the same Windows line endings as the existing models. I'm leaving out a Pydantic `schemas/application.py` because it belongs with the endpoint task. Say so if you want it now.

Follow-up once job postings exist

1. In `models/application.py`, change `job_id` to `Column(Integer, ForeignKey("jobs.id", ondelete="RESTRICT"), nullable=False, index=True)`. Optionally add `job = relationship("Job")`.
2. Run `002_add_applications_job_fk.sql` once on Supabase. `create_all` won't alter the existing table, so step 1 alone doesn't change the live database.

This assumes the table will be named `jobs` with an integer `id`, as the existing conventions suggest.

For the later apply-endpoint task, three things must come with it:

- The resume ownership check.
- Mapping a duplicate-application database error to a 409. The database rule is what stops a double-click from creating two rows.
- The resume-delete guard from decision 1.

Verification

I won't touch Supabase or `.env`. In a scratch environment outside the repo, using the SQLAlchemy version already pinned in `requirements.txt`, I'll generate the Postgres DDL from the model and diff it against `001`. I'll also test the unique, CHECK and delete rules on a throwaway in-memory database.

Reply "approved" to go with all the recommendations, or name the ones to change (for example "1B, 5B"). Tell me too if you want the optional Pydantic schema.

Prompt 2:

app has been updated since then. Please generate me a prompt for the agent that builds this part under the hood of the application. Then, the prompt should check the most recent version of the folder to see if it can be implemented properly. If it can do a full implementation. If not, keep the functionality built in and it'll be called and implemented when the software catches up

Result 2:

I'll check what the Sprint 2 backend looks like now, so the prompt matches the current code, and read the project notes first.

I rewrote the plan as a prompt for the building agent. It checks the latest state of the folder first and picks one of two modes:

- **Full:** if a backend jobs model exists, it builds the complete table with the job foreign key.

- **Deferred:** if not, it builds everything else and leaves the job connection ready to switch on. The duplicate-application rule works in both modes.

I read "functionality built in" as the data layer only, so no endpoints.

Task: build the data layer for job applications (issue #25, parent #24 / US-06)

Role

You are a senior backend engineer working in this repo (CareerConnect: FastAPI + SQLAlchemy 2.0 + Pydantic v2 + PostgreSQL on Supabase; React 19 + TypeScript + Vite). The code for this sprint is in `Sprint 2/backend`. Follow the existing conventions exactly, keep changes minimal, and work autonomously from start to finish. Stop and ask me only if something you find contradicts this prompt in a way that changes the design.

Goal

Add the database layer that records a job application: it links a job posting, an applicant (user) and the resume they chose, plus a submission timestamp and a status.

Product context (for understanding only, do NOT build it): every applicable job listing will show two buttons. "Select" opens a vertical menu of the user's resumes; picking one replaces the label with that resume's name. "Apply" is washed out and unclickable until a resume is selected. The selected-but-not-applied state exists only on screen and is never saved; only the final choice is saved when Apply is pressed.

The job postings feature may not exist in the backend yet. So: first check the latest state of the folder (Step 0). If jobs exist, do a full implementation (Mode A). If not, build everything that can be built now so it is ready to be connected the moment jobs exist (Mode B).

Hard rules

1. Data side only. No API endpoints or routers, no Pydantic schemas, no frontend changes, no sample/seed data.

2. Never open, print, search inside or modify `.env` or any secrets file (e.g. `Sprint 2/backend/.env`). Never run anything against the shared Supabase database. Only ever use a LOCAL Postgres (localhost / 127.0.0.1) when a database is needed.

3. No new libraries or tools. Do not edit `requirements*.txt`, `package*.json` or `.github/`. If you think one is needed, stop and ask.

4. Do not change how users, resumes or profiles work. The only existing file you may edit is `Sprint 2/backend/models/__init__.py` (one import line + one `__all__` entry). Do NOT edit `routers/*`, `models/user.py`, `models/resume.py`, `models/profile.py`, `app.py` or `database.py`. If something else seems necessary, stop and ask, and say why.

5. Git is read-only except `git fetch`. No pull, merge, rebase, checkout, stash, reset, commit or push. Leave your changes uncommitted in the working tree.

6. Line endings: `git status` lists ~30 files as modified only because the working tree is CRLF while the git index is LF (confirm: `git diff --ignore-cr-at-eol --stat` is empty). Ignore that; do not "fix" or touch those files. Edit existing files in place keeping their current endings; create new files with LF.

Step 0: Preflight (read-only; finish before writing any code)

Run from the repo root and keep the evidence for your report.

a. `git fetch --all --prune`, then `git branch --show-current`, `git log --oneline -5`, `git status --short` (ignoring the line-ending noise).

b. Is the remote ahead? `git log --oneline HEAD..origin/main -- "Sprint 2/backend"` and `git diff --name-only HEAD...origin/main -- "Sprint 2/backend/models"`. If origin/main is ahead AND those commits add or change a job model, STOP without writing code and tell me to pull first (a full implementation is possible after that).

c. Does anything already exist? `ls "Sprint 2/backend/models"`, `grep -rn "__tablename__" "Sprint 2/backend/models"`, and search `*.py` (excluding `venv/`) for `Application` / `applications`. If an applications model, table or migration already exists, STOP and report; do not overwrite.

d. Re-read the current conventions, which changed recently: `models/user.py`, `models/resume.py` (now has `is_primary`), `models/profile.py`, `models/__init__.py`, `database.py`, `app.py` (`Base.metadata.create_all` on startup), `tests/conftest.py`, `pytest.ini`, `.github/workflows/ci.yml`. Known facts to confirm: legacy `Column(...)` style; FKs written

`ForeignKey("users.id")` with no ON DELETE; `datetime.datetime.utcnow` defaults; one model per file re-exported from `models/__init__.py`; tests run only against a LOCAL Postgres (conftest refuses anything else); CI runs pytest on Postgres 16.

e. Migration tooling: is there an `alembic` entry in `requirements*.txt`, or an `alembic/` folder / `alembic.ini`? If yes, write an Alembic revision instead of the plain SQL files below (install nothing). If no (expected), use plain SQL files in `Sprint 2/backend/migrations/`.

f. Jobs readiness, checked in the CURRENT checkout only. All three must hold: (1) a model file in `models/` defining the jobs table (expected `__tablename__ = "jobs"`; record the real class and table name); (2) it is imported in `models/__init__.py`; (3) it has an integer primary key `id` (the frontend `Job` interface uses `id: number`). A job model that exists only on another unmerged branch does NOT count; just mention which branch. (As of the last fetch I know of, `origin/Job-Search-Listing` only has frontend code: a `Job` interface with a numeric id, and sample data.)

Step 1: Pick the mode, then say so

Mode A (full implementation) if (f) holds. Mode B (deferred connection) otherwise. Print a short preflight summary (mode + evidence) and continue; no need to wait for me.

Step 2: Locked design (both modes)

Create `Sprint 2/backend/models/application.py`, table `applications`, in the same style as the existing models: legacy `Column(...)` declarative style, `from database import Base`, an "Owner:" docstring, explanatory comments in the same voice, and one-direction-only `relationship()`s (no `back_populates`, so `User` and `Resume` stay untouched).

| Column | Definition |

|---|---|

| `id` | Integer, primary key, index=True |

| `user_id` | Integer, FK `users.id` with ON DELETE CASCADE, NOT NULL |

| `job_id` | Integer, NOT NULL, index=True (FK to the jobs table: see the modes) |

| `resume_id` | Integer, FK `resumes.id`, NOT NULL, no ON DELETE rule (= NO ACTION) |

| `submitted_at` | DateTime, NOT NULL, Python `default=datetime.datetime.utcnow` AND `server_default=text("timezone('utc', now())")` |

| `status` | String(20), NOT NULL, `default="applied"` AND `server_default="applied"` |

Constraints:

- `UniqueConstraint("user_id", "job_id", name="uq_applications_user_id_job_id")`: one applicant can never have two applications to the same posting. Must be live in both modes.

- `CheckConstraint` named `ck_applications_status`, built from a module-level `APPLICATION_STATUSES = ("applied", "interview", "offered", "rejected")` (lowercase snake_case, like `role`'s `job_seeker`). Keep that tuple as the single source of truth; the future Pydantic schema will import it.

- Relationships: `applicant = relationship("User")` and `resume = relationship("Resume")` (named like `Resume.owner`).

Locked decisions (change only if I tell you to):

- Resume deleted: the database refuses to delete a resume that any application uses. Use the default NO ACTION (omit `ondelete`, like every existing FK) rather than an explicit RESTRICT. Same protection for a direct delete, but it will not misfire if a future delete-account flow removes a user's applications and resumes in one statement.

- User deleted: their applications are deleted with them (CASCADE). Note in a comment that `resumes.user_id` has no ON DELETE rule today, so a user who has resumes still cannot be deleted; a future delete-account flow must remove applications, then resumes, then the user, in that order.

- Job deleted (later): NO ACTION. Postings with applicants get closed/archived, not deleted.

- The DB only guarantees the resume EXISTS, not that it belongs to the applicant. Do not add a composite FK (it would alter the existing `resumes` table). State in the model docstring that the future apply endpoint must reuse `_get_owned_resume` from `routers/resume.py`.

- Also record in the docstring two facts about existing behaviour that the apply task must handle: (1) `delete_resume` removes the file from disk BEFORE deleting the row, so once applications

exist it needs a pre-check that returns 409 before `os.remove`; (2) `replace_resume` (PUT) swaps the file behind the same resume row, so an application points at the resume row, not a frozen copy of the file.

Step 3A: Mode A (jobs exist), full implementation

- `job_id = Column(Integer, ForeignKey("<jobs table>.id"), nullable=False, index=True)` using the real table name, plus `job = relationship("<JobClass>")` (one-direction).

- `migrations/001_create_applications.sql` creates the complete table including the job FK. No `002` file.

- Add the import to `models/__init__.py`.

Step 3B: Mode B (jobs don't exist yet), ready to connect

Do NOT create a jobs model, table or stub, and do NOT declare a ForeignKey to a table that isn't registered (SQLAlchemy would raise `NoReferencedTableError` at startup and break the app).

- `job_id = Column(Integer, nullable=False, index=True)`, with a greppable comment block starting `TODO(job-fk)` containing: the exact replacement line (`Column(Integer, ForeignKey("jobs.id"), nullable=False, index=True)`), the optional `job = relationship("Job")`, and the instruction to run `migrations/002_add_applications_job_fk.sql` once afterwards (because `create_all` never alters an existing table).

- `migrations/001_create_applications.sql`: the full table without the job FK.

- `migrations/002_add_applications_job_fk.sql`: one `ALTER TABLE applications ADD CONSTRAINT applications_job_id_fkey FOREIGN KEY (job_id) REFERENCES jobs (id);`, with a header saying "run only after the jobs table exists" and a commented check query for orphan job_ids.

- Add the import to `models/__init__.py` so `create_all` builds the table and the uniqueness rule is live now. No endpoint can write rows yet, so no orphan `job_id` can appear before the FK is added.

Step 4: SQL files

Plain SQL, safe to paste into the Supabase SQL editor, each with a header comment (what it does, when to run it). Use `CREATE TABLE IF NOT EXISTS` / `CREATE INDEX IF NOT EXISTS`. Generate the exact DDL from the model (`sqlalchemy.schema.CreateTable` / `CreateIndex` compiled with the `postgresql` dialect) and make the SQL match it: same column types, defaults, nullability and constraint/index names, with explicit names (`applications_user_id_fkey`, `applications_resume_id_fkey`, `ix_applications_id`, `ix_applications_job_id`, ...). Running the file and letting `create_all` build the table must give the same table.

Step 5: Tests

Add `Sprint 2/backend/tests/test_applications.py`, following `conftest.py` (use the `make_user` fixture and `database.SessionLocal` directly; insert `Resume` rows with dummy `filename`/`filepath` and no real files; no endpoints). Always `db.rollback()` after an expected `IntegrityError`, and delete the applications/resumes a test created in a `finally`. Cover:

1. A new application gets status `applied` and a non-null `submitted_at`.

2. The same `(user_id, job_id)` twice is rejected; a different job for the same user, and the same job for a different user, are accepted.

3. A non-existent `user_id` or `resume_id` is rejected (and a non-existent `job_id` in Mode A).

4. A status outside `APPLICATION_STATUSES` is rejected.

5. Deleting a resume that an application uses is refused and leaves the resume and application intact.

6. Deleting the applicant user deletes their applications. Use a Core `delete()` statement, and to avoid the resumes FK blocking the delete, point the application at a resume owned by a different user (the DB allows it; the endpoint will forbid it).

7. Mode A only: deleting a job that has applications is refused.

In Mode B, job ids are arbitrary integers; add a comment at the top of the file saying that the job cases of tests 3 and 7 get added once the FK is connected.

Step 6: Verify (no Supabase, no `.env`)

1. `python -m py_compile` on your new files, and an import check of `models` using a throwaway local-looking URL such as `DATABASE_URL=postgresql+pg8000://x:y@localhost/z` (the engine connects lazily, so no connection is opened). Use the project's existing venv (`Sprint 2/backend/venv`, `Scripts/` on Windows) and install nothing.

2. Render the DDL from the model and diff it against `001_create_applications.sql`.

3. Only if a LOCAL Postgres is available (Docker already installed, or something on localhost:5432 that is not Supabase): from `Sprint 2/backend` run `DATABASE_URL=postgresql+pg8000://postgres:postgres@localhost:5432/careerconnect_test pytest -v` for the WHOLE suite (new tests plus existing ones, to prove nothing regressed). Never run pytest without that explicit localhost `DATABASE_URL`. If no local Postgres exists, don't install one; say the tests were not run and give the exact command.

4. `git status --short` and `git diff --ignore-cr-at-eol --stat` must show only your new files and the one-line addition to `models/__init__.py`.

Step 7: Final report (max 25 lines, no recap of steps)

- Mode chosen and the evidence behind it.

- Files created / edited (the one existing file, and why).

- Anything you deviated from, and why.

- What was verified and what could not be (e.g. tests not run).

- Follow-up checklist. Mode B: (1) swap the `job_id` line per the `TODO(job-fk)` comment; (2) run `002_add_applications_job_fk.sql` once. Always, for the future apply-endpoint task: reuse `_get_owned_resume`; map a duplicate-application `IntegrityError` to HTTP 409; add the resume-delete guard before `os.remove`; decide what to do about `replace_resume` changing a resume already used in an application; add Pydantic schemas (`ApplicationOut`, status `Literal` from `APPLICATION_STATUSES`).

- Open questions for me, if any.

Today it will almost certainly land in the deferred mode. I found no backend jobs model on your `main` (it's at the same commit as `origin/main`, as of your last fetch). The only job work is on

`origin/Job-Search-Listing` and is frontend only: a `Job` interface with a numeric `id`, plus sample data. That's why the prompt assumes `job_id` is an integer.

The prompt differs from the plan I gave you earlier in a few ways. It assumes you accepted all my recommendations; they are listed under "Locked decisions" if you want to change any.

- **Resume delete rule:** I changed explicit RESTRICT to the default NO ACTION, which every existing foreign key already uses. A direct delete is still refused. The difference is that NO ACTION won't misfire if a future delete-account flow removes a user's applications and resumes in one statement.
- **Database-side defaults:** `submitted_at` and `status` now have defaults in the database as well as in Python. This means the SQL file and `create_all` produce the same table, and rows inserted by hand still get defaults.
- **Tests added:** the repo now has pytest and CI against a local Postgres. The agent writes data-layer tests for the unique, foreign key, status and delete rules, and runs them only if it finds a local Postgres. Delete Step 5 if you don't want them.

New finding to flag: `PUT /resume/{id}` swaps the file behind the same resume row. An application therefore points at the resume row, not a frozen copy of what was submitted. I did not try to fix this. The prompt records it as something the apply-endpoint task must decide.

About 30 files show as modified in `git status` only because your working tree uses CRLF line endings while git stores LF. The prompt tells the agent to ignore them. You may want to check your autocrlf setting before your next commit. I stayed out of `.env` again.

Prompt 3:

Backend for task 3 has been created. Please generate a new prompt for the agent for task 6.3

Result 3:

Task 6.3 (issue #27, parent US-06 #24): Build the apply flow in the UI — resume selection, confirmation, success message

Role

You are a senior frontend engineer working in this repo (CareerConnect: React 19 + TypeScript + Vite frontend in `Sprint 2/frontend`; FastAPI + SQLAlchemy backend in `Sprint 2/backend`). Follow the existing conventions exactly, keep changes minimal, and work autonomously from

start to finish. Stop and ask me only if something you find contradicts this prompt in a way that changes the design.

Goal

Let a job seeker apply to a job from its listing card by choosing one of their resumes, confirming, and seeing a success message.

On every applicable listing (the viewer is a job seeker, the job is open, and they haven't applied to it yet) there are two buttons side by side:

1. **Select**, blue (about #3d49cc in our mockup). It opens a vertical menu listing the job seeker's resumes by filename; there can be any number. Picking one closes the menu and replaces the "Select" label with that resume's filename. It can be reopened to change the choice.

2. **Apply**, purple (about #a24ba3). By default it is washed out and unclickable. Once a resume is selected it returns to its normal, brighter colour.

3. Pressing Apply opens a **confirmation**: "Apply to {job title} at {company} using {resume filename}?" with Cancel / Confirm.

4. Confirm submits the application. On success, show a **success message** and replace the two buttons with a non-interactive "Applied" state.

The "selected but not applied yet" state lives only in component state. It is never saved (no API call, no storage). Only Confirm saves anything.

The job listing cards may not exist on `main` yet. So: first check the latest state of the folder (Step 0). If the listing exists, do a full implementation wired into it (Mode A). If not, build everything that can be built now so it can be dropped into the listing the moment it lands (Mode B).

Backend contract (already merged; do NOT change the backend)

Confirm each item by reading `Sprint 2/backend/routers/application.py` and `schemas/application.py` before coding.

- `GET /api/resume/mine` → `Resume[]` (`id, user_id, filename, uploaded_at, is_primary`). `listMyResumes(token)` already exists in `src/api.ts`.

- `POST /api/applications`, body `{ "job_id": number, "resume_id": number }` → 201 `{ message, application: ApplicationOut }`. Always send `resume_id`. The backend falls back to the primary resume when it's missing, but the UI must never rely on that.

- `GET /api/applications/mine` → `ApplicationOut[]` (`id, job_id, job_title, company, resume_id (nullable), resume_filename (nullable), status, applied_at, updated_at`).

- Errors:

  - 409: already applied ("You have already applied to this job.")

  - 400: job closed, or the user has no resume

  - 403: not a job seeker, or not your resume

  - 404: job or resume not found

  - 401: bad or expired token

  - 422: invalid body

Hard rules

1. Frontend only. Don't touch `Sprint 2/backend`. Don't build the job listing page, search or filters, and don't add sample or seed data.

2. Never open, print, search inside or modify `.env` or any secrets file. Never start the backend against the shared Supabase database (its `.env` points there) and never submit real applications to it.

3. No new dependencies or tools. Don't edit `package.json`, `package-lock.json` or `.github/`. If you think a library is needed, stop and ask.

4. Don't modify my uncommitted local work, which is a theme / dark-mode change in progress:

   - modified: `index.html`, `src/main.tsx`, `src/index.css`, `src/styles/auth.css`, `src/components/Login.tsx`, `Register.tsx`, `Sidebar.tsx`, `icons.tsx`

- new, untracked: `src/theme.ts`, `src/components/ThemeToggle.tsx`

Re-check `git status` at the start and treat anything listed there as off-limits. Don't touch the git stash either.

5. Git is view-only except `git fetch`. No pull, merge, rebase, checkout, stash, reset, commit or push. Leave your changes uncommitted for me to review.

6. If you must edit an existing file, keep it to the smallest addition and justify it in your report. Preserve each file's current line endings, and create new files with LF.

Step 0: Preflight (read-only; finish before writing code)

a. Run `git fetch --all --prune`, `git status --short`, `git log --oneline -5` and `git log --oneline HEAD..origin/main`. If origin/main is ahead AND those commits touch `src/components/tabs/JobDashboardTab.tsx` or add job listing code or a `Job` type, STOP without writing code and tell me to pull first.

b. Re-read the current conventions:

- `src/api.ts`: the `request` helper, `errorMessage`, and the "matches schemas/…" comments.

- `src/components/Dashboard.tsx`: how `token` reaches tabs. `JobDashboardTab` currently gets only `user`.

- `src/components/tabs/JobDashboardTab.tsx`.

- `src/components/ResumeManager.tsx`: state, loading and error patterns, comment voice.

- `src/index.css`: the `--cc-*` tokens, light on `:root`, dark under `:root[data-theme="dark"]` (my local work is moving them). Also the `.cc-error`, `.cc-success`, `.cc-btn-secondary` and `.cc-card*` classes.

- `tsconfig.json` and `eslint.config.js`.

c. Listing readiness, checked in the CURRENT checkout only: does `JobDashboardTab.tsx` render a list of job cards, from real data or sample data, using a `Job` type in `api.ts`? A listing that exists only on another branch does NOT count; just mention it. (As of my last check, `origin/Job-Search-Listing` has the cards plus a `GET /api/jobs` search endpoint, unmerged.)

d. Does anything for this task already exist (e.g. `ApplyControls`, `submitApplication`, an apply stylesheet)? If yes, STOP and report; don't overwrite.

Step 1: Pick the mode, then say so

Mode A (full, wired into the listing) if (c) holds. Mode B (built in, ready to drop in) otherwise. Print a short preflight summary (the mode and the evidence behind it) and continue; no need to wait for me.

Step 2: Locked design (both modes)

API: additions to `src/api.ts`:

- Types `ApplicationOut` and `ApplicationSubmitResponse`, mirroring `schemas/application.py`, with the same "Matches schemas/…" comment style.

- `submitApplication(jobId, resumeId, token)` and `listMyApplications(token)`, written like `listMyResumes`.

- The UI must tell a 409 apart from other errors without matching message text. Make the smallest change that allows it: an exported `ApiError extends Error` with a `status` field, thrown by `request` instead of a plain `Error`. Existing callers are unaffected because it is still an `Error` with the same message; say that in a comment.

Component `src/components/ApplyControls.tsx`:

- Props: `job: { id: number; title: string; company: string }`, `token: string`, `alreadyApplied?: boolean`, `onApplied?: (application: ApplicationOut) => void`.

- Resumes: fetch lazily the first time the menu opens, and keep them for the component's lifetime.

  - Show loading, and an error with a retry.

  - Empty state: "You don't have any resumes yet. Upload one in your Profile tab."

  - Show a small "Primary" tag on the primary resume, but do NOT preselect it.

- Long filenames: truncate with an ellipsis and put the full name in `title`.

- Apply: use the native `disabled` attribute until a resume is selected, plus a visible hint linked with `aria-describedby` ("Select a resume first").

- Confirmation: a native `<dialog>` opened with `showModal()`, which gives focus handling and Esc for free with no library. Cancel returns focus to Apply.

- Submitting:

  - Disable Select, Apply and Confirm, and show "Applying…".

  - Guard with a ref so a double click can't send two requests.

- Success:

  - Show a message in an element with `role="status"` (e.g. "Application sent to {company}.").

  - Switch to the "Applied" state.

  - Call `onApplied`.

- Errors: shown inline in an element with `role="alert"`. Keep the selection so the user can retry, except:

  - 409: switch to the "Applied" state and show the backend's message.

  - 400 "no longer accepting applications": show the message and keep Apply disabled.

  - 401: show the message. The app has no token-refresh flow.

- If `alreadyApplied` is true, render only the "Applied" state.

- Menu accessibility:

  - The Select trigger has `aria-haspopup="listbox"` and `aria-expanded`.

  - The menu is a `role="listbox"` with `role="option"` items.

  - ↑/↓ moves, Enter selects, Esc closes and returns focus to Select, and clicking outside closes.

  - Visible focus rings, and touch targets at least 44px tall.

- Write comments in the same explanatory voice as `ResumeManager.tsx`.

Styles: new file `src/styles/apply.css`, imported by the component. Do NOT edit `index.css`.

- Scope everything under a `.cc-apply` root class. Define local custom properties there for the Select and Apply fills (normal, hover, and washed-out disabled).

- Put dark-theme overrides under `:root[data-theme="dark"] .cc-apply`.

- "Washed out" means a lighter, desaturated fill with readable text, not just `opacity`.

- White text on the normal Select and Apply fills must reach at least 4.5:1 contrast. Adjust the mockup colours slightly if needed, and report the final values.

- Reuse the existing `--cc-*` tokens for text, borders, success, error and radius wherever possible.

Locked decisions (change only if I tell you to):

- "Confirmation" in the issue title means a confirm step BEFORE submitting. The success message comes after.

- Status comes from the backend's `ApiError.status`, never from string matching.

- Resume choice is explicit: no preselection, even though the primary resume exists.

- One `listMyApplications` request per listing view, never one per card.

Step 3A: Mode A (listing cards exist), full implementation

- `Dashboard.tsx`: pass `token={token}` to `<JobDashboardTab>`. One prop; this is the only reason to touch that file.

- `JobDashboardTab.tsx`: for job seekers only, call `listMyApplications` once and build a `Set` of applied `job_id`s.

  - Render `<ApplyControls>` in each card whose job is open (if the `Job` type has `is_open`).

  - Pass `alreadyApplied`, and an `onApplied` that adds the id to the set.

  - Recruiters see no apply controls.

- If the cards still use sample data whose ids aren't real jobs in the database, wire it anyway and flag it in the report. Submitting would return 404 until real jobs load.

Step 3B: Mode B (no listing yet), ready to drop in

- Do NOT edit `JobDashboardTab.tsx` or `Dashboard.tsx`. Don't create a listing, a job type or sample jobs.

- At the top of `ApplyControls.tsx`, add a short "HOW TO USE" comment containing:

  - the exact snippet a listing card needs: `<ApplyControls job={job} token={token} alreadyApplied={appliedIds.has(job.id)} onApplied={…} />`

  - the two wiring steps from Mode A (pass `token` into the tab; fetch `listMyApplications` once).

Step 4: Verify

1. In `Sprint 2/frontend`, run `npm run lint` and `npm run build`, which is what CI runs. `eslint.config.js` only lints `*.js`/`*.jsx` and `vite build` does not type-check, so also run `npx tsc --noEmit -p .` (TypeScript is already a dev dependency). All three must pass with no new warnings. If something fails only because of my uncommitted theme files, report it; don't fix those files.

2. Don't run the backend against Supabase. If a LOCAL Postgres is available, you may run the backend with an explicit localhost `DATABASE_URL` and click through the flow. Otherwise skip it and give me a manual test checklist covering:

   - Select changes the label; Apply enables.

   - The confirm dialog appears; Cancel works.

   - The success message shows and the card switches to Applied.

   - A duplicate shows Applied.

   - A user with no resumes sees the empty state.

   - Everything works keyboard-only.

   - Dark theme looks right.

3. `git status --short` and `git diff --ignore-cr-at-eol --stat` must show only your new files, the justified additions to `api.ts`, and in Mode A the edits to `Dashboard.tsx` and `JobDashboardTab.tsx`, alongside my pre-existing local changes.

Step 5: Final report (max 25 lines, no recap of steps)

- Mode chosen and the evidence behind it.

- Files created and edited, with why for each edit to an existing file.

- Final colour values and contrast ratios.

- Anything you deviated from, and why.

- What was verified and what couldn't be.

- Mode B: the exact steps to wire it into the listing once `Job-Search-Listing` merges.

- Open questions for me, if any.

AI-Suggested Content: AI suggested a design for the applications table, with options and a recommendation for each question I had (what happens when a resume or user is deleted, and which statuses to use). It then wrote a prompt for the coding agent for issue #25 and another for Task 6.3, the Select and Apply buttons with a confirmation step and a success message. Each prompt tells the agent to check the latest version of the project first, so it can do a full implementation if the app is ready and a prepared version if it isn't.

Validation: AI read the real project before writing each prompt, and I gave it a screenshot of issue #27 so the scope came from the actual ticket. I checked that my rules (no secrets, no new libraries, nothing pushed to GitHub, existing features unchanged) are written into the prompts. I couldn't test the result end to end yet, because the job listing page isn't merged and the backend tests need a local database.

Decision: Accepted. The prompts work whether or not the rest of the app has caught up, and I review the agent's changes before committing anything.

Reflection: Building the database and server first, then the UI, let each prompt rely on what already existed. Claude also caught problems I hadn't asked about, like how deleting or replacing a resume could affect applications. What I created with AI seems fine for now, but I'm always monitoring the code after every new change to see if the code runs into any future problems that need resolving or any changes/improvements can be made.

Responsible Person: Ethan Fadlon 40314189